A3 Mock e Observabilidade

Equipe

Gustavo Gutierres Champam - RA 223645
Gustavo Oliveira Camargo - RA 236024

Objetivo

Finalizar os jogos da semana anterior, persistir a identificação dos vencedores e somente depois enviar SMS e e-mail. Implementamos a solução com Java 17, Spring Boot 3.3.3, Mockito/JUnit, JPA/H2, JaCoCo, Prometheus e Grafana.

Resultado

O build clean install passou. Nos testes, o projeto refatorado atingiu 100% de linhas e ramos no JaCoCo, sem excluir classes. Testes de ordem, falha de persistência, datas, API e persistência foram executados. As métricas foram coletadas e visualizadas em processos reais.

Escopo e origem

Partimos do mock_spring.zip fornecido na atividade. O pacote lab é uma refatoração focada na regra ampliada, e não uma cópia que preserva todos os endpoints e modelos antigos. A implementação não realiza pagamentos nem envia SMS e e-mail real; SmsLog é um adapter didático que registra o envio. A revisão em dupla deve conferir especialmente o limite transacional e os cenários de falha antes da apresentação.

Atualização SMS e e-mail

Regra ampliada

Implementamos SMS e e-mail após a confirmação da persistência do lote. Se o banco falhar, nenhum dos canais é chamado. O envio é sequencial: primeiro SMS, depois e-mail; falha no SMS interrompe o fluxo antes do e-mail. Não há retry automático, entrega garantida ou provedor externo configurado: SmsLog e EmailLog são adapters simulados para o estudo com mocks.

Validação em 2 de novembro

13 testes aprovados e 100% de linhas e ramos no JaCoCo. Os relatórios HTML e os logs acompanham esta versão.

Evidências históricas

As capturas PNG no final registram a execução de 26/10/2026. O build.log e o JaCoCo HTML foram atualizados em 02/11/2026.

Prática 1 Entendimento dos mocks

Classes de negócio originais

Jogo mantém descrição, data e resultados; Participante identifica competidor; Resultado associa participante e métrica; Juiz determina maior/menor pontuação; FinalizaJogo seleciona e finaliza jogos; Pagamento modela valor e data, mas não implementa liquidação. CriadorDeJogo auxilia a montagem dos cenários. JogoDao e JogoDaoFalso são infraestrutura, não regras de negócio.

Problemas do DAO real

JogoDao abre MySQL no construtor. Um teste passa a depender de driver, schema, credenciais, rede e dados existentes. Escritas deixam estado compartilhado e tornam falhas difíceis de atribuir. Há SQL manual e fechamento de recursos para manter. O fake original usa lista estática, contaminando testes, e não possui a mesma hierarquia/contrato do DAO concreto exigido por FinalizaJogo.

Complexidade do DAO falso

Um fake precisa reproduzir consulta, atualização, identificadores e isolamento sem imitar incorretamente transações do banco. Quanto mais SQL e regras forem reproduzidos nele, maior a manutenção e o risco de divergência. A porta SalvarLote permite substituir infraestrutura sem herança de DAO que conecta no construtor.

Injeção de dependência

O construtor de FinalizaJogo recebe SalvarLote e Sms; o teste escolhe mocks e o Spring injeta adapters na aplicação. Injeção por construtor torna dependências obrigatórias e visíveis; setter permite troca opcional, mas admite objeto incompleto; campo é conciso, porém esconde dependências e dificulta instanciação sem framework. Preferimos construtor.

Prática 1 Estado e comportamento

Stub e mock no código

Em ControllerTest, when(servico.finalizar(...)).thenReturn(2) é um stub: controla a resposta para verificar o HTTP. O mesmo objeto Mockito é usado como mock quando verify(servico).finalizar(...) confere a delegação. Em FinalizaJogoTest, doThrow(...) é um stub de falha da persistência. Assertions verificam valores/estado; verify e InOrder verificam interações.

Finalidade de verify

verify confirma se uma interação ocorreu com os argumentos e a quantidade esperados. verifyNoInteractions(sms) prova ausência de qualquer chamada no colaborador após erro do banco. InOrder verifica que salvar o lote antecede as chamadas de SMS e e-mail. Apenas verificar duas chamadas isoladas não demonstra sua ordem.

Correções em relação à base

A base selecionava qualquer data com pelo menos sete dias, inclusive jogos de anos atrás. Nesta solução, semana anterior é segunda-feira inclusiva até a segunda-feira atual exclusiva, conforme LocalDate fornecida ao caso de uso. Data é explícita para testes determinísticos. Desempate por nome em ordem alfabética é uma premissa didática; jogo sem resultados aborta a preparação do lote.

Como reproduzir

Com Java 17: executar mvnw.cmd clean install (Windows) ou ./mvnw clean install (Linux). O Maven roda JUnit e verifica cobertura. Em seguida, java -jar target/a3-jogos-1.0.0.jar --server.port=8083. POST /jogos/finalizar?hoje=2026-10-26 recebe requisicao.json. Evidências de teste e execução estão em evidencias; relatório HTML completo em evidencias/jacoco/index.html.

Prática 2 Persistência antes do SMS e e-mail

Regra implementada

FinalizaJogo monta o lote de jogos elegíveis e seus vencedores antes de qualquer gravação. PersistenciaJpa.salvar usa saveAllAndFlush dentro de @Transactional. O proxy Spring confirma a transação antes de retornar ao chamador. Somente após esse retorno ocorre lote.forEach(sms::enviar). FinalizaJogo não é transacional, pois uma transação externa atrasaria o commit até depois do SMS e e-mail.

Casos de teste

Sucesso com dois jogos: uma gravação do lote antes dos dois SMS e e-mail. Falha da gravação: exceção e nenhuma interação com SMS e e-mail. Datas anterior ao intervalo, atual ou futura: zero persistência e zero SMS e e-mail. Lista vazia: nenhum efeito. Jogo sem resultado: aborta o lote inteiro. Empate: desempate determinístico. Falha do adapter SMS e e-mail: ocorre depois da gravação, sem alegar rollback do SMS e e-mail.

Integração e limite transacional

IntegrationTest usa o adapter JPA real e consulta o vencedor salvo dentro do mock de SMS e e-mail; verifica também que nenhuma transação permanece ativa no momento do envio. O teste de falha do banco é simulado com Mockito. H2 valida mapeamento e SQL no lab; não substitui teste do banco de produção.

Cobertura e limites

JaCoCo mede 31 linhas e 6 ramos cobertos no projeto refatorado, com zero linhas/ramos perdidos. Cobertura não prova ausência de bugs. Requisições repetidas podem reenviar SMS e e-mail; uma falha no segundo SMS e e-mail pode deixar entrega parcial. A regra exigida de não enviar antes de salvar é cumprida, mas idempotência e retry durável exigiriam outbox e controle persistente em uma evolução.

Prática 3 Integração e API

MockMVC aplicado

@WebMvcTest carrega a camada MVC com DispatcherServlet, conversão JSON e validação de parâmetros, sem abrir servidor HTTP. @MockBean substitui FinalizaJogo; a métrica usa SimpleMeterRegistry. ControllerTest verifica HTTP 200 com resposta 2 e HTTP 400 para data inválida. É rápido e isola contratos do controller; não demonstra rede, proxy ou banco real.

DataJpaTest aplicado

@DataJpaTest carrega entidades, repositories e banco embarcado H2, com rollback por teste. RepositoryTest salva, faz flush, limpa o EntityManager e busca novamente o vencedor. A limpeza evita confirmar apenas o objeto do cache de primeiro nível. Isso detecta problemas de mapeamento e persistência que um mock de repository não detecta.

Teste de repository

O repository real verifica que id e vencedor podem ser gravados e recarregados. Seu valor é testar contrato de dados e integração JPA, não verificar somente se save foi chamado. Para PostgreSQL/MySQL em produção, acrescentar Testcontainers e os índices/constraints específicos do banco.

Teste de API e teste integrado

ControllerTest contém comentários e exemplo aplicado de API com mocks. IntegrationTest sobe o contexto completo e comprova o limite de transação com JPA real e SMS e e-mail simulado. A execução do JAR e requisições HTTP reais complementa o teste MVC, oferecendo evidência do processo da aplicação e do Actuator.

Prática 4 Observabilidade

Fluxo das métricas

Spring Boot Actuator/Micrometer expõe /actuator/prometheus. Prometheus coleta a cada cinco segundos e armazena séries. Grafana consulta Prometheus, sem buscar métricas diretamente na API. Provisionamos quatro painéis técnicos e um contador de jogos finalizados.

O que observar

up{job="a3-jogos"} indica sucesso da coleta; não prova que a regra de negócio funciona. Taxa de requests mostra tráfego; soma de tempo dividida por quantidade mostra latência média em segundos; jvm_memory_used_bytes com area=heap mostra memória; jogos_finalizados_total conta finalizações que terminaram o caso de uso com sucesso. Sem tráfego, a média pode ficar sem dados.

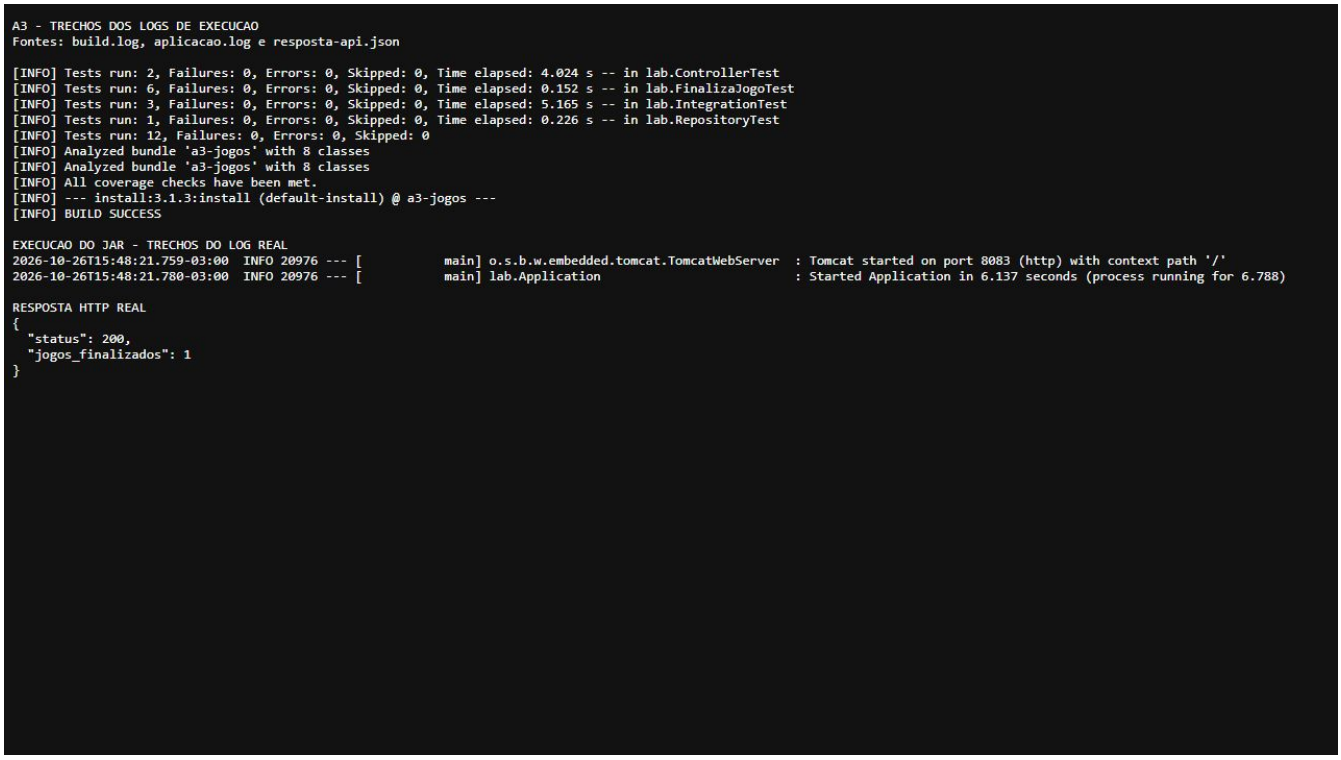
Evidência executada

As capturas grafana.png e prometheus.png registram métricas reais. A execução usou processos nativos Windows, com Prometheus na 9093 e Grafana na 3003. compose.yml e provisioning permitem repetir o laboratório com Docker, mas esse Compose não foi executado nesta máquina.

Reprodução com Docker

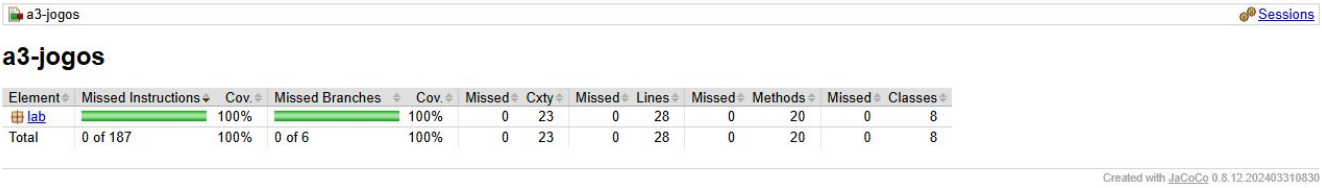
Executar mvnw.cmd clean install e docker compose up -d --build. API: localhost:8083; Prometheus: localhost:9093; Grafana: localhost:3003. No Grafana do Compose, acesso inicial admin/admin do laboratório. Enviar requisicao.json, aguardar scrapes e abrir o dashboard A3. O banco H2 é em memória, portanto dados são descartados quando o processo termina.

Execução Java JUnit e Maven



Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

JaCoCo com cobertura integral


a3-jogos

Sessions

a3-jogos

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
lab		100%		100%	0	23	0	28	0	20	0	8
Total	0 of 187	100%	0 of 6	100%	0	23	0	28	0	20	0	8

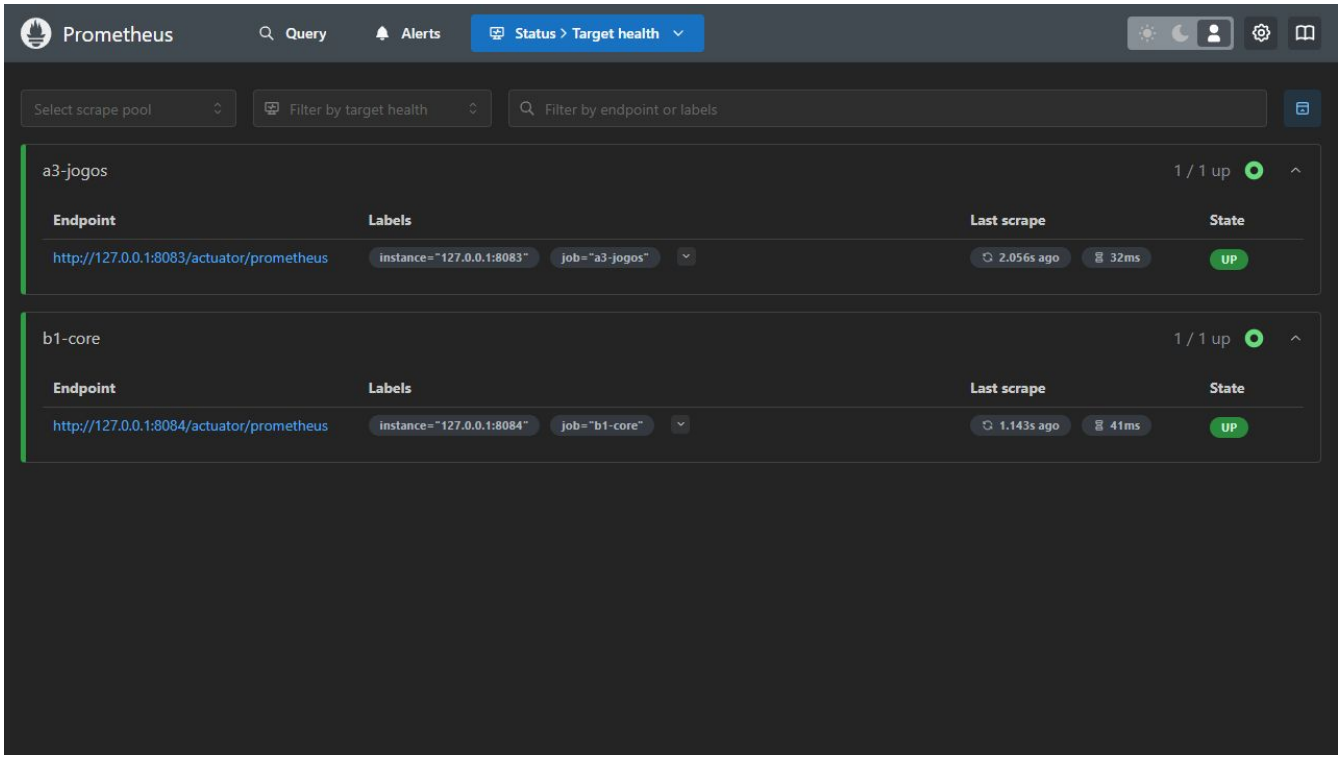
Created with JaCoCo 0.8.12.202403310830

Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

Resultado do build atualizado

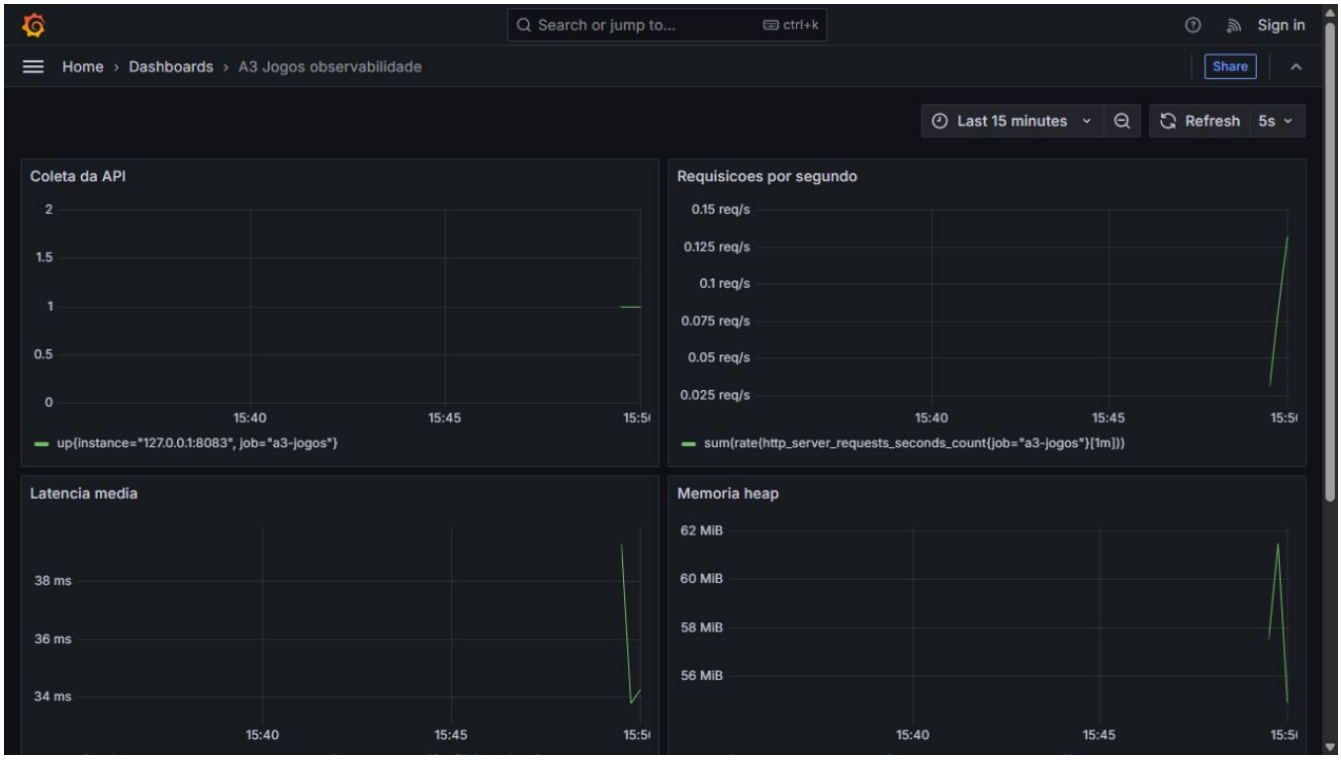
13 testes; zero falhas, erros ou testes ignorados. JaCoCo: 210 instruções, 31 linhas, 6 ramos e 9 classes cobertos. Maven clean install concluído com BUILD SUCCESS. O relatório completo acompanha a entrega em HTML e CSV.

Prometheus com coleta ativa



Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

Dashboard Grafana



Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

Métrica de negócio após execução



Captura da execução local em 26 de outubro de 2026. O arquivo original em alta resolução acompanha o pacote em evidencias.

Referências e rastreabilidade

Enunciados e projetos fornecidos pela disciplina.

Spring Boot 3.3 - Testing Spring Boot Applications:
https://docs.spring.io/spring-boot/3.3/reference/testing/spring-boot-applications.html

Mockito - InOrder: https://site.mockito.org/javadoc/current/org/mockito/InOrder.html

JaCoCo - check goal: https://www.jacoco.org/jacoco/trunk/doc/check-mojo.html

C4 Model: https://c4model.com/

Prometheus overview: https://prometheus.io/docs/introduction/overview/

As métricas do relatório são da execução local. Jenkins/Compose e capacidade de produção não foram validados. Os arquivos Markdown mantêm o conteúdo editável e os comandos necessários para repetir os experimentos.